BitChat Android Security Analysis

Hiba Zahreddine, Sebastien Larinier

1 Scope and Methodology

The analysis presented in this section was conducted on BitChat Android version 1.7.2, released under the package `com.bitchat.android`. The APK was obtained directly from the project's GitHub releases page.

- **Version:** 1.7.2
- **SHA-256:** `39f7fce76931da5db7f0eebd0d17b5bb957e948e1be145851eeb186ee183d27f`
- **Analysis period:** May – June 2026
- **Source:** `https://github.com/permissionlesstech/bitchat-android/releases/download/1.7.2/bitchat-universal-release.apk`

The scope of this analysis is limited to the Bluetooth mesh communication layer of the application. The Nostr relay integration, Tor, and any other internet-based transport layers were considered out of scope, as the application's primary security claim centers on its offline mesh capability.

Several observations about the codebase are worth noting before presenting the findings:

- The Android branch of the application was developed through AI code generation prompted from the IOS-version with the contributor publicly stating that he is not an android developer.
- A significant volume of dead and unused code is present, several functions are defined multiple times across different file.
- Github issues were not being addressed correctly with most of them ignored or silently fixed.
- Removal of some security mechanism code that was not clarified, we tried contacting the contributor in may to ask why his code was removed but we didn't get an answer.
- The IOS-version that is maintained by a different developer, is more carefully implemented , and was silently fixed after an external audit (this was concluded from the commits messages)
- A significant number of the critical findings were addressed in BitChat 2.0 , which was recently released(August 2026).

2 Analysis Process

The analysis was originally planned as a two-phase process: static analysis followed by dynamic analysis. The dynamic phase was ultimately de-prioritized, as the volume and severity of findings identified through source code review alone was sufficient to conclude that the application's stated security properties did not hold in practice.

Testing the App as users.
We started by testing the app using 2 android devices in the same room, several issues were clear: both devices' batteries were quickly drained and overheated during use. The QR code verification mechanism meant to verify a user was non-functional, scanning a peer's QR code produced no result. Most significantly, establishing a private end-to-end encrypted chat between the two devices proved impossible despite repeated attempts, while broadcast messages (which are transmitted in cleartext)functioned without issue. The application also claimed in its interface to support password-protected encrypted channels, but no such functionality was accessible or operational. These usability issues made us question if the app was actually used as advertised during the protests.

Static Analysis.
Since the source code is publicly available we used **Android Studio**[1] to navigate the code,this allowed us to easily map the code between classes and call hierarchy navigations and dependencies, it also allowed us to see what functions were used and how they were called. Additionally the APK was submitted to **MobSF**[3] (Mobile Security Framework), an automated static analysis platform that performs manifest analysis, permission auditing, dependency checks, and pattern-based detection of common implementation weaknesses. To avoid the accidental analysis of unused code, we also used **JADX**[2] a decompiler that reconstructs Java source from compiled Android bytecode, to work with the released version and verify it against the source code.

3 Findings :

The following findings were identified through the analysis. Each finding is tagged as a Design and/or Implementation fault. The findings are ordered within each pillar by severity.

3.1 Trust Establishment:

The trust establishment pillar revealed the most significant concentration of Design - level failures in the application. The following findings were identified:

- **Independent signing identities with no cross-binding[Design - Critical]:** The Ed25519 signing key used for mesh announcements and the Noise static public key used for session encryption are generated and stored independently with no cryptographic proof-of-possession binding between them. An attacker can replay a victim's Noise public key inside an ANNOUNCE message signed with their own Ed25519 key; on first contact the forged binding is accepted as legitimate, after which the genuine victim's subsequent ANNOUNCE is rejected as an unauthorized key change attempt.*This finding was partially addressed in BitChat 2.0.1*
- **Multiple paths silently destroy the fingerprint anchor[Design + Implementation - Critical]:** The stale peer cleanup mechanism removes fingerprint anchors for peers not seen within the last 3 minutes. The nickname-collision eviction logic additionally replaces an

existing peer entry and its fingerprint anchor if a new peer announces the same nickname and the original peer has not been seen in ten seconds. These two paths can be combined into the following attack chain:

1. An attacker observes the victim's nickname from mesh traffic, which requires no authentication
2. The attacker waits for the victim to be silent for 10 seconds, or for the stale cleanup timer to fire
3. The attacker broadcasts an ANNOUNCE with the victim's nickname and their own key
4. `addOrUpdatePeer()` evicts the victim's peer entry and fingerprint mapping
5. The attacker completes a Noise handshake using their own key
6. `storeFingerprintForPeer()` finds no existing entry and stores the attacker's key as ground truth
7. The attacker is now cryptographically trusted as the victim's identity with no indication to the local user

Note that this attack chain was not empirically tested in practice and requires further validation. *This finding was addressed in BitChat 2.0.1*

- **Session drop before validation [Design - High]:** The handshake handler drops an existing established session before validating whether the new handshake request is legitimate. By design, any peer can force-terminate an active session by sending a new handshake request with the same peerID, even with an invalid key, resulting in a denial of connectivity. *This finding was addressed in BitChat 1.7.3*

- **In-memory only fingerprint anchor resets on every restart[Design - High]:** The fingerprint binding introduced to protect against key substitution is held exclusively in memory and is never persisted to storage on application startup. Every restart resets all peers to first-contact status, meaning the trust established during a previous session provides no protection in the next one. In a trust-on-first-use model, an anchor that disappears on every restart is equivalent to no trust model at all. *This finding was addressed in BitChat 2.0.1*

- **Circular identity binding[Design - High]:** BitChat derives the peerID from the first eight hexadecimal characters of the SHA-256 hash of the user's static public key, then treats the binding between the peerID and this truncated hash as a verification mechanism. Binding a value to a truncated version of itself provides no independent security guarantee and reduces the identifier's entropy to the point where collision risk becomes a practical concern.

- **Circular signature validation[Design - Medium]:** The ANNOUNCE message carries the Ed25519 signing public key against which subsequent MESSAGE signatures are verified. However, the ANNOUNCE message itself is unauthenticated. Accepting a signing key from an unverified broadcast and using it to verify messages from the same session is circular: the verification is only as trustworthy as the unverified claim it is built upon. This means that any peer on the mesh can impersonate any other peer simply by announcing a signing key of their choice in an ANNOUNCE message, with no mechanism to detect or prevent the substitution.

- **Key verification mechanism non-functional[Implementation - Medium]:** The QR code scanning interface, which constitutes the application's only user-facing mechanism for out-of-band key verification, was non-functional during testing: scanning a peer's QR code

produced no result. This means the sole mechanism by which a user could detect a man-in-the-middle attack was inoperable in the evaluated version, regardless of whether the underlying trust model was sound. *This finding was addressed in BitChat 2.0.3*

- **Channel password validation always returns true[Implementation - Medium]:** The function responsible for validating the password of password-protected channels returns `true` unconditionally in the evaluated version, while the application's README and interface claim that password-protected channels are a supported security feature. The feature is advertised as implemented and functional but provides no actual access control. *The feature remains documented but unimplemented.*
- **Deprecated secure storage [Implementation - Low]:** The app uses `EncryptedSharedPreferences` to store the static private key. This library is deprecated on Android, and its security model stores the encryption key on the device itself, meaning a rooted device can access both the encryption key and the data it protects.

3.2 Conversation Security:

The conversation security pillar presents a more mixed picture, with several sound design choices undermined by significant implementation failures.

Verified positives:

- The application correctly uses the Noise protocol framework for one-to-one encrypted sessions, a well-established and formally analyzed construction.
- SecureRandom is used throughout for key generation with no weak random number generator identified.
- AEAD is applied with verify-before-decrypt, constant-time tag comparison, nonce wraparound enforcement, and session key zeroization on destruction.

Findings:

- **Session key material logged in release build[Implementation - High]:** The Noise session key material is written to Android's logcat logging system. *This finding was addressed in BitChat 2.0.*
- **Decrypted message content logged in release build[Implementation - High]:** The first thirty characters of every decrypted message and the sender's peerID is written to logcat and not stripped in the release build. *This finding was addressed in BitChat 2.0.*
- **Absent signature accepted as valid[Implementation - High]:** SecurityManager.verifySignature() returns true when packet.signature == null. An attacker can strip the signature field (which is transmitted in plaintext in the packet) and the verification step is bypassed entirely. *This finding was addressed in BitChat 2.0.*
- **Replay protection window shift is directionally incorrect[Implementation - High]:** The replay protection mechanism in the Noise session implementation contains a directional error in its bit-shift logic. *This finding was addressed in BitChat 2.0.*
- **Encryption scope does not cover channel communications[Design - Medium]:** Broadcast channel messages are transmitted without encryption. The application's interface and documentation do not clearly distinguish encrypted private messages from unencrypted broadcast communications, creating a false impression of comprehensive encryption.

3.3 Transport Security:

Although a Bluetooth radio capture device was not available for dynamic traffic interception, the packet structure could be analyzed statically through mapping the functions that create the packets, these were the findings:

- **TTL-based relay with no upper bound check[Design - High]:** shouldRelayPacket() relays any packet with TTL ¿= 4 unconditionally, on the assumption that high TTL indicates an important message. Since TTL is a plaintext field controllable by the sender, an attacker can set any TTL value to force relay, enabling network flooding. *This finding was addressed in BitChat 2.0.*
- **Duplicate packet detection exception[Design - High]:** the code that is supposed to verify if a packet has already been relayed contains an exception for ANNOUNCE messages arriving from direct neighbors at maximum TTL, a field that can be controlled by an attacker, allowing continuous re-announcement of a substituted signing key to bypass deduplication. *This finding was addressed in BitChat 2.0.*
- **Relay directory updated unsigned from a third-party source[Implementation - High]:** The Nostr relay directory is automatically updated daily from a third-party GitHub repository over an unsigned and unpinned connection.
- **Metadata leakage in relayed packets[Design - Medium]:** While message payloads are encrypted, several fields in every relayed packet are transmitted in plaintext: type, senderID, recipientID, timestamp, TTL, flags, and signature.

3.4 Summary Table

Finding	Pillar	Layer	Severity	Status
Independent signing identities with no cross-binding	Trust	Design	Critical	Addressed (2.0.1)
Multiple paths silently destroy the fingerprint anchor	Trust	Design + Impl	Critical	Addressed (2.0.1)
Session key material logged in release build	Conversation	Implementation	High	Addressed (2.0.1)
Decrypted message content logged in release build	Conversation	Implementation	High	Addressed (2.0.1)
Absent signature accepted as valid	Conversation	Implementation	High	Addressed (1.6.0)
TTL-based relay with no upper bound check	Transport	Design	High	Addressed (2.0.1)
Duplicate packet detection exception	Transport	Design	High	Addressed (2.0.1)
Relay directory updated unsigned	Transport	Implementation	High	**Not addressed**
Session drop before validation	Trust	Design	High	Addressed (1.7.3)
In-memory fingerprint anchor resets on restart	Trust	Design	High	Addressed (2.0.1)
Replay protection non-functional	Conversation	Implementation	High	Addressed (1.7.3)
Circular identity binding	Trust	Design	High	**Not addressed**
Circular signature validation	Trust	Design	Medium	**Not addressed**
Key verification mechanism non-functional	Trust	Implementation	Medium	Addressed (2.0.3)
Channel password always returns true	Trust	Implementation	Medium	**Not addressed**
Encryption scope excludes channel communications	Conversation	Design	Medium	**Not addressed**
Metadata leakage in relayed packets	Transport	Design	Medium	**Not addressed**
Deprecated secure storage	Trust	Implementation	Low	**Not Addressed**

Table 1: Summary of findings with current status as of BitChat Android v2.0.3

References

[1] Android studio. https://developer.android.com/studio.

[2] Jadx github. https://github.com/skylot/jadx.

[3] Mobile security testing framework. https://mobsf.org/.